Durable Execution (Enabled by Persistence)

What is Durable Execution?

- Technique where workflow **saves progress** at key points (checkpoints)
- Allows **pause** and **resume** exactly where it left off — even after:
  - Long delays (days/weeks)
  - System crashes
  - Human review / approval
  - Transient errors (LLM timeout, rate limit, network issue)

Why Durable Execution matters in LangGraph

- Long-running agents / multi-step reasoning
- **Human-in-the-loop** workflows (inspect, edit, approve)
- Production reliability: survive failures without restarting from scratch
- Avoid expensive re-computation of previous steps

If you use a checkpointer → you already have durable execution

- MemorySaver, SQLiteSaver, PostgresSaver, etc.
- Every super-step → checkpoint saved Resume by calling***invoke(None, config)*** with same ***thread_id***

Durable Execution — Core Requirements & Determinism

Core Requirements (must-have)

1. Enable **persistence** → pass a **checkpointer** when compiling graph
2. Always provide **thread_id** in config (unique per conversation/run)
3. Wrap **non-deterministic** or **side-effect** operations inside **@task** (or separate nodes)

Determinism & Consistent Replay

When resuming:

- LangGraph does **NOT** continue from exact line of code
- It **replays** from start of the interrupted **node** (or entrypoint)
- Same code runs again up to failure point

Therefore → must guarantee same outcome on replay:

- Wrap **side effects** (API calls, file writes, logging, sending emails) in **@task**
- Wrap **non-deterministic** ops (random(), UUID generation, current time) in **@task**
- Prefer **idempotent** operations where possible (use idempotency keys, check-if-exists)

Durable Execution — Durability Modes & Resuming

Durability Modes:

- "exit" → persist only when graph fully exits (fastest, least durable)
- "async" → persist asynchronously while next step runs (good balance)
- "sync" → persist synchronously before next step (most durable, slowest)

When to use each mode:

- Prototyping / short runs → "exit" or "async"
- Production long-running agents with human-in-loop → "sync"
- Very high throughput → "async" or "exit" + accept small risk

Resuming in Practice — Common Scenarios:

- Human-in-the-loop → interrupt() → resume with updated state via Command
- Error recovery → exception occurs → re-run same thread_id with None input → Graph resumes from last successful checkpoint
- Long pause → user comes back days later → same thread_id → resume

Time Travel (Enabled by Persistence)

What is Time Travel?

Ability to go back to any previous checkpoint :

- Replay exactly the same execution
- Change something (state / input / tool result) and continue differently
- Create a new branch in history (fork)

Why is Time Travel useful?

- Understand why the agent made a certain decision
- Debug why something went wrong (wrong tool, bad reasoning, hallucination)
- Try different paths without restarting from beginning
- Compare multiple "what if" versions of the same conversation/agent run
- Support human-in-the-loop correction and retry

LangGraph Time Travel = git for agent runs → checkout any previous state, change something, commit new branch

Time Travel — Three Common Use Cases

1. **Understand Reasoning:** See exactly how a good answer was reached
    - Replay step-by-step what the agent “thought”
    - Inspect intermediate decisions, tool calls, LLM outputs
    - Discover clever paths the model took that you didn’t expect
    - Explain to stakeholders / team / users: “This is why the answer makes sense”
    - Very valuable for chain-of-thought, ReAct, multi-hop reasoning agents
2. **Debug Mistakes:** Find precisely where and why the agent failed
    - Jump back to the moment before the wrong action happened
    - See bad tool arguments, hallucinated facts, wrong assumptions
    - Identify prompt leakage, context overflow, tool misuse
    - Much faster than adding logs everywhere or re-running many times
    - Typical pattern: go back 1–2 steps → read messages → spot the error
3. **Explore Alternatives:** Quickly test “what if” scenarios without restarting everything
    - Change one thing and continue from there:
        - Fix hallucinated fact
        - Give better / different tool result
        - Rewrite last LLM message
        - Try different prompt style mid-flow
        - Simulate human correction
    - Compare multiple futures from the same starting point
    - Very powerful for prompt engineering, tool tuning, recovery flows

Time Travel — How It Works

Core Mechanism:

- Graph must use a checkpointer (MemorySaver, SQLiteSaver, PostgresSaver, etc.)
- Every super-step creates a checkpoint
- You can list history → graph.get_state_history(config)
- You choose one checkpoint (by checkpoint_id)
- (Optional) Modify state with update_state()
- Resume from that point → graph.invoke(None, config_with_checkpoint_id)

Important Keywords:

- thread_id → one conversation / one agent run lineage
- checkpoint_id → specific snapshot inside that thread
- fork → new branch created after update_state + resume
- get_state_history() → shows past states in reverse chronological order

Typical debugging / exploration flow

- Run graph normally (with persistence enabled)
- Look at final result → not good?
- ***list(graph.get_state_history(config))***
- Pick interesting checkpoint (usually states[1] or states[2])
- (Optional) ***graph.update_state(selected_config, {"key": new_value})***
- Resume: ***graph.invoke(None, new_config)***

Minimal realistic pattern (most common in debugging)

initial → node1 → node2 → node3 (bad) ← current end
↑
└─ update_state() here → node3' → node4' (better)

```python
# After bad result
states = list(graph.get_state_history(config))

# Usually we want state before last LLM call
bad_state = states[1]

# Option A: just replay same
graph.invoke(None, bad_state.config)

# Option B: fix something and continue
new_config = graph.update_state(
    bad_state.config,
    {"messages": [... corrected messages ...]}
    # or {"last_tool_result": "... fixed data ..."}
)

final = graph.invoke(None, new_config)
```

When Time Travel is especially powerful

- Agent loops / multi-step reasoning gone wrong
- Tool calling with bad arguments or hallucinations
- Human wants to correct facts mid-conversation
- Comparing prompt variations without re-running everything
- Recovering from rate-limits / API errors in long runs

Important notes

- You need **persistence** enabled (checkpointer)
- invoke(None, config) = resume from last checkpoint of that config
- update_state() creates **new checkpoint_id** but same thread_id
- History is linear until you branch → branches are created on demand